CRYPTOGRAPHY LABORATORY

Record of Programs

NAME: THANVEER T

REGISTER NUMBER: 192424107

COURSE CODE: CSA5112

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

EX. NO: 1

CAESAR CIPHER

Aim:

To write a C program to implement the Caesar Cipher, a classical substitution technique, for encrypting a given plaintext using a fixed key value.

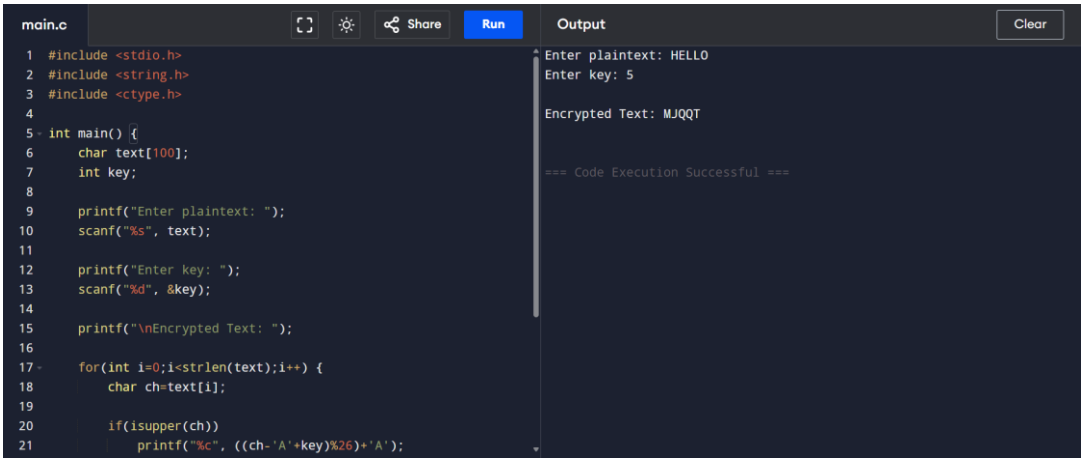
Algorithm:

1. Start the program.
2. Read the plaintext and the key (shift value) from the user.
3. Traverse each character of the plaintext.
4. If the character is an uppercase letter, shift it circularly within 'A' to 'Z' by the key value.
5. If the character is a lowercase letter, shift it circularly within 'a' to 'z' by the key value.
6. If the character is not an alphabet, leave it unchanged.
7. Display the resulting encrypted text.
8. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char text[100];
    int key;

    printf("Enter plaintext: ");
    scanf("%s", text);
    printf("Enter key: ");
    scanf("%d", &key);

    printf("\nEncrypted Text: ");
    for (int i = 0; i < strlen(text); i++) {
        char ch = text[i];
        if (isupper(ch))
            printf("%c", ((ch - 'A' + key) % 26) + 'A');
        else if (islower(ch))
            printf("%c", ((ch - 'a' + key) % 26) + 'a');
        else
            printf("%c", ch);
    }
    printf("\n");
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Caesar Cipher encryption technique was written, executed and the output was verified successfully.

EX. NO: 2

MONOALPHABETIC CIPHER

Aim:

To write a C program to implement the Monoalphabetic Cipher, which encrypts a plaintext using a fixed substitution key derived from a permutation of the alphabet.

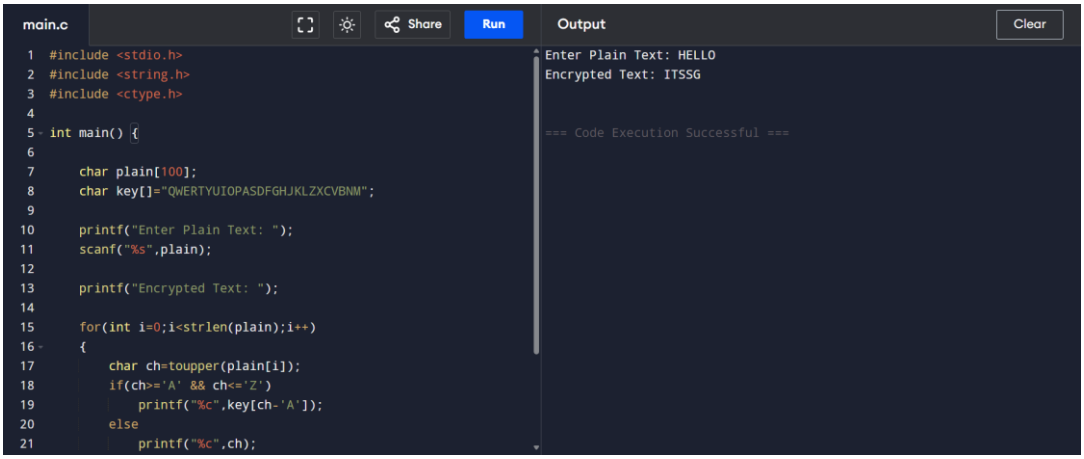
Algorithm:

1. Start the program.
2. Define a substitution key array containing a permutation of the twenty-six letters of the alphabet.
3. Read the plaintext from the user.
4. Convert each character of the plaintext to uppercase.
5. For every alphabetic character, replace it with the character present at the corresponding position in the substitution key.
6. Leave non-alphabetic characters unchanged.
7. Display the encrypted text.
8. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char plain[100];
    char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";

    printf("Enter Plain Text: ");
    scanf("%s", plain);

    printf("Encrypted Text: ");
    for (int i = 0; i < strlen(plain); i++) {
        char ch = toupper(plain[i]);
        if (ch >= 'A' && ch <= 'Z')
            printf("%c", key[ch - 'A']);
        else
            printf("%c", ch);
    }
    printf("\n");
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Monoalphabetic Cipher using a substitution key was written, executed and the output was verified successfully.

EX. NO: 3

POLYALPHABETIC CIPHER (VIGENERE CIPHER)

Aim:

To write a C program to implement the Vigenere Cipher, a polyalphabetic substitution technique that uses a repeating keyword to encrypt the plaintext.

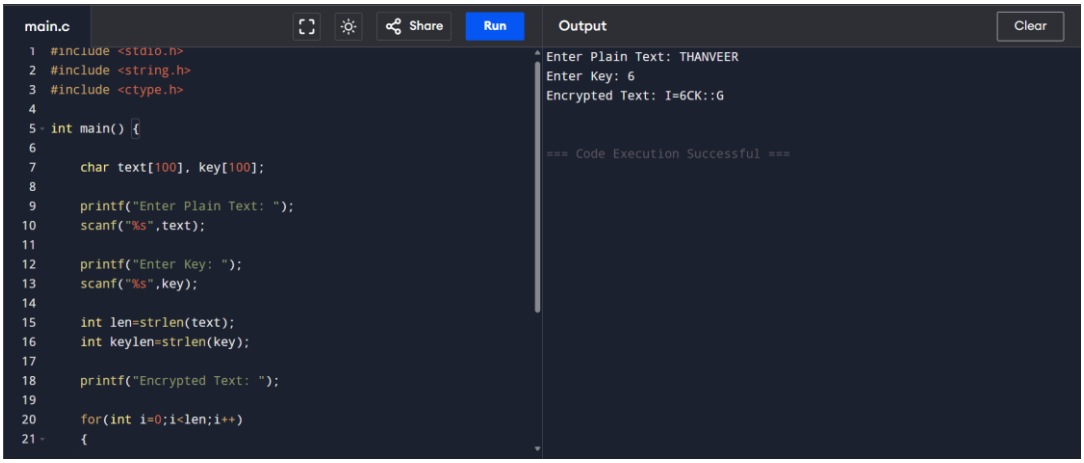
Algorithm:

1. Start the program.
2. Read the plaintext and the keyword from the user.
3. Determine the length of the plaintext and the length of the keyword.
4. For every character in the plaintext, convert both the plaintext character and the corresponding keyword character (repeated cyclically) to uppercase.
5. Compute the cipher character using the relation (plaintext value + key value) mod 26.
6. Display the resulting encrypted text.
7. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char text[100], key[100];

    printf("Enter Plain Text: ");
    scanf("%s", text);
    printf("Enter Key: ");
    scanf("%s", key);

    int len = strlen(text);
    int keylen = strlen(key);

    printf("Encrypted Text: ");
    for (int i = 0; i < len; i++) {
        char ch = toupper(text[i]);
        char k = toupper(key[i % keylen]);
        char enc = ((ch - 'A') + (k - 'A')) % 26 + 'A';
        printf("%c", enc);
    }
    printf("\n");
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Vigenere Cipher, a polyalphabetic substitution technique, was written, executed and the output was verified successfully.

EX. NO: 4

HILL CIPHER (2 X 2 MATRIX)

Aim:

To write a C program to implement the Hill Cipher, a polygraphic substitution technique, using a 2 x 2 key matrix for encryption.

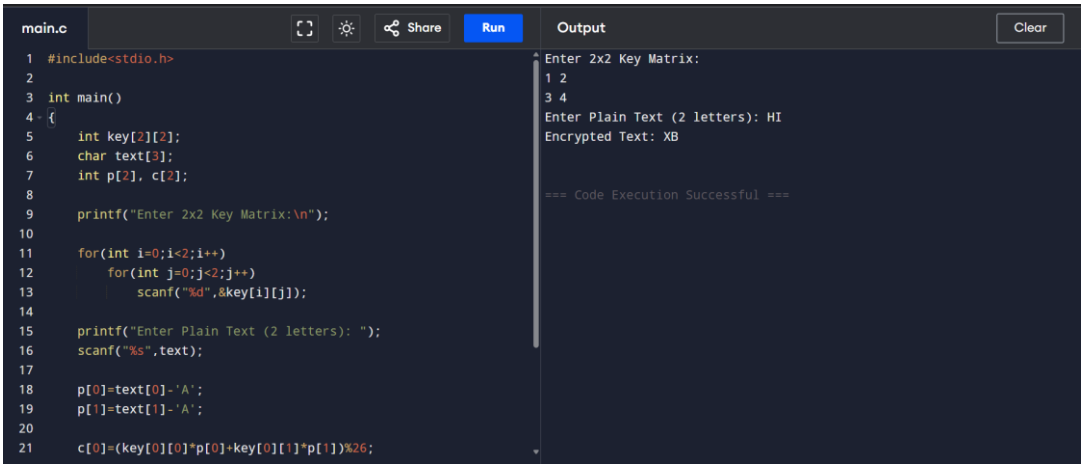
Algorithm:

1. Start the program.
2. Read the elements of the 2 x 2 key matrix from the user.
3. Read two letters of the plaintext and convert them into their numeric equivalents (A = 0, B = 1, ... , Z = 25).
4. Multiply the key matrix with the plaintext vector to obtain the cipher vector.
5. Reduce each element of the cipher vector modulo 26.
6. Convert the resulting numeric values back into their corresponding letters.
7. Display the encrypted text.
8. Stop the program.

Program:

```
#include <stdio.h>

int main() {
    int key[2][2];
    char text[3];
    int p[2], c[2];

    printf("Enter 2x2 Key Matrix:\n");
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            scanf("%d", &key[i][j]);

    printf("Enter Plain Text (2 letters): ");
    scanf("%s", text);

    p[0] = text[0] - 'A';
    p[1] = text[1] - 'A';

    c[0] = (key[0][0] * p[0] + key[0][1] * p[1]) % 26;
    c[1] = (key[1][0] * p[0] + key[1][1] * p[1]) % 26;

    printf("Encrypted Text: ");
    printf("%c%c\n", c[0] + 'A', c[1] + 'A');
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Hill Cipher using a 2 x 2 key matrix was written, executed and the output was verified successfully.

EX. NO: 5

PLAYFAIR CIPHER

Aim:

To write a C program to implement the Playfair Cipher, a digraphic substitution technique that encrypts pairs of letters using a 5 x 5 key matrix.

Algorithm:

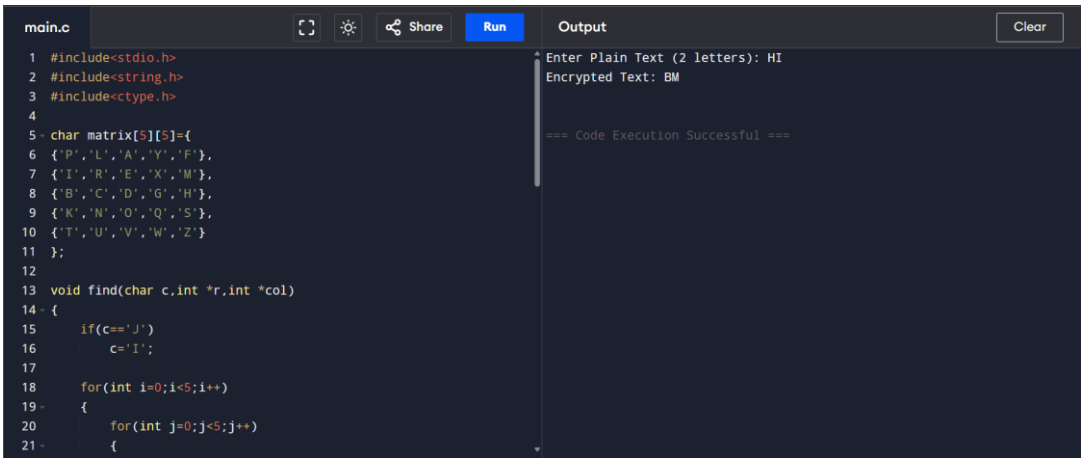
1. Start the program.
2. Construct a 5 x 5 key matrix using the keyword "PLAYFAIR", combining the letters I and J into a single cell.
3. Read a pair of plaintext letters from the user.
4. Locate the row and column positions of both letters within the key matrix.
5. If both letters lie in the same row, replace each with the letter immediately to its right, wrapping around the row if required.
6. If both letters lie in the same column, replace each with the letter immediately below it, wrapping around the column if required.
7. If the letters lie in different rows and columns, replace each letter with the one in its own row but in the column of the other letter (rectangle rule).
8. Display the resulting encrypted pair of letters.
9. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5] = {
    {'P', 'L', 'A', 'Y', 'F'},
    {'I', 'R', 'E', 'X', 'M'},
    {'B', 'C', 'D', 'G', 'H'},
    {'K', 'N', 'O', 'Q', 'S'},
    {'T', 'U', 'V', 'W', 'Z'}
};

void find(char c, int *r, int *col) {
    if (c == 'J')
        c = 'I';
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (matrix[i][j] == c) {
                *r = i;
                *col = j;
```

```
            }
        }
    }
}

int main() {
    char text[100];
    printf("Enter Plain Text (2 letters): ");
    scanf("%s", text);

    int r1, c1, r2, c2;
    find(toupper(text[0]), &r1, &c1);
    find(toupper(text[1]), &r2, &c2);

    if (r1 == r2) {
        printf("Encrypted Text: %c%c\n",
            matrix[r1][(c1 + 1) % 5],
            matrix[r2][(c2 + 1) % 5]);
    } else if (c1 == c2) {
        printf("Encrypted Text: %c%c\n",
            matrix[(r1 + 1) % 5][c1],
            matrix[(r2 + 1) % 5][c2]);
    } else {
        printf("Encrypted Text: %c%c\n",
            matrix[r1][c2],
            matrix[r2][c1]);
    }
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Playfair Cipher using a 5 x 5 key matrix was written, executed and the output was verified successfully.

EX. NO: 6

RAIL FENCE CIPHER

Aim:

To write a C program to implement the Rail Fence Cipher, a transposition technique that rearranges plaintext characters in a zig-zag pattern across a defined number of rails.

Algorithm:

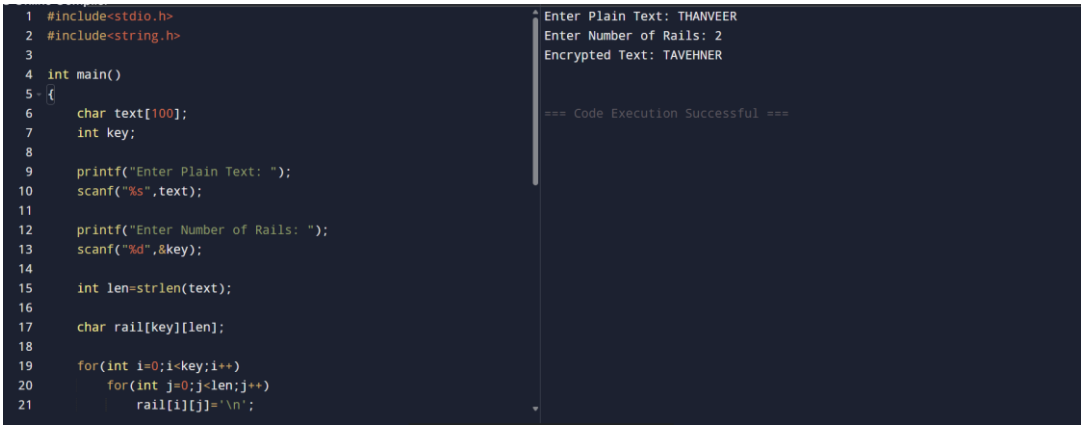
1. Start the program.
2. Read the plaintext and the number of rails from the user.
3. Create a two-dimensional matrix with rows equal to the number of rails and columns equal to the length of the plaintext.
4. Traverse the plaintext and place each character into the matrix, moving down and up across the rails in a zig-zag manner.
5. Read the matrix row by row, skipping empty cells, to obtain the encrypted text.
6. Display the resulting encrypted text.
7. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char text[100];
    int key;

    printf("Enter Plain Text: ");
    scanf("%s", text);
    printf("Enter Number of Rails: ");
    scanf("%d", &key);

    int len = strlen(text);
    char rail[key][len];

    for (int i = 0; i < key; i++)
        for (int j = 0; j < len; j++)
            rail[i][j] = '\n';

    int row = 0;
    int dir = 1;
    for (int i = 0; i < len; i++) {
        rail[row][i] = text[i];
        if (row == 0)
            dir = 1;
        else if (row == key - 1)
```

```
            dir = -1;
        row += dir;
    }

    printf("Encrypted Text: ");
    for (int i = 0; i < key; i++)
        for (int j = 0; j < len; j++)
            if (rail[i][j] != '\n')
                printf("%c", rail[i][j]);
    printf("\n");
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Rail Fence Cipher transposition technique was written, executed and the output was verified successfully.

EX. NO: 7

COLUMNAR TRANSPOSITION CIPHER

Aim:

To write a C program to implement the Columnar Transposition Cipher, a transposition technique that rearranges plaintext characters by reading a filled matrix column by column.

Algorithm:

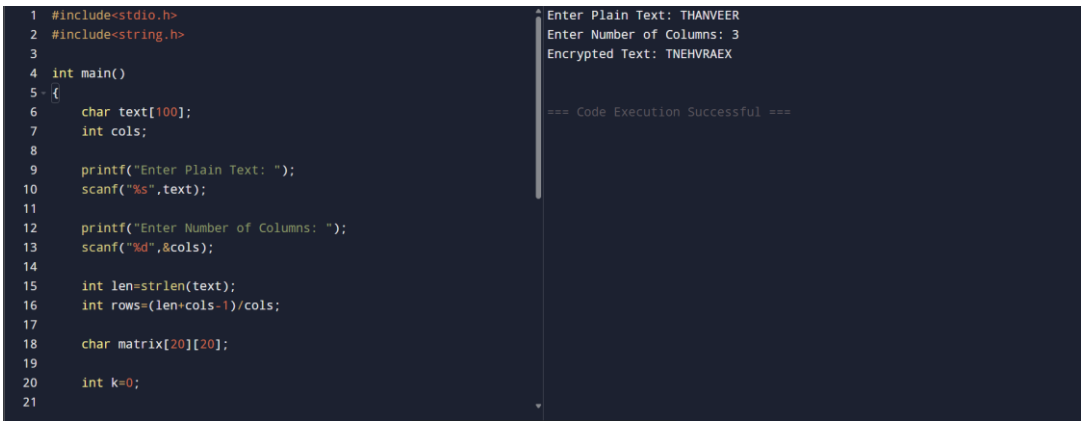
1. Start the program.
2. Read the plaintext and the number of columns from the user.
3. Calculate the number of rows required to accommodate the plaintext.
4. Fill the matrix row-wise with the characters of the plaintext, padding any remaining cells with the letter 'X'.
5. Read the matrix column by column to generate the encrypted text.
6. Display the resulting encrypted text.
7. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char text[100];
    int cols;

    printf("Enter Plain Text: ");
    scanf("%s", text);
    printf("Enter Number of Columns: ");
    scanf("%d", &cols);

    int len = strlen(text);
    int rows = (len + cols - 1) / cols;
    char matrix[20][20];
    int k = 0;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (k < len)
                matrix[i][j] = text[k++];
            else
                matrix[i][j] = 'X';
        }
    }

    printf("Encrypted Text: ");
```

```
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++)
            printf("%c", matrix[i][j]);
    }
    printf("\n");
    return 0;
}
```

Output:



Result:

Thus a C program to implement the Columnar Transposition Cipher was written, executed and the output was verified successfully.